

Master Project Review & Unit Testing Reference Guide

Exhaustive Code Walkthrough, Spring Boot Fundamentals, DTO & Exception Architecture, Git Mastery, and 89-Test Unit Testing Suite

PROJECT	ARCHITECTURE	TEST SUITE	TARGET REVIEW
LuxeFlats Rental Marketplace	Spring Boot 3 + Microservices	89 Tests (100% Pass Rate)	Project Viva / Tech Evaluation

CHAPTER 1: LUXEFLATS MICROSERVICES ARCHITECTURE & SYSTEM FLOW

LuxeFlats is a distributed real-estate rental platform engineered with Spring Boot 3 microservices adhering to the **Database-per-Service** pattern. Each service operates in complete domain isolation with its own persistent schema, communicating through synchronous RESTful contracts.

Service	Port	Core Responsibilities	Database & Entities	Inter-Service Dependencies
API Gateway	8080	Central reverse proxy, unified entry point, routing requests to downstream services, CORS filter.	None (Stateless routing)	Routes to Ports 8081, 8082, 8083, 8084
Auth Service	8081	User registration, login, BCrypt password hashing, JWT generation and validation, role-based RBAC (TENANT, OWNER, ADMIN).	H2 DB (authdb) Entity: User	Standalone (Provides JWT to all clients)
Property Service	8082	Property listings CRUD, search filters (city, type, price, furnishing), pipe-delimited image storage, availability toggle.	H2 DB (propertydb) Entity: Property	Invoked by Booking Service to verify property existence & owner identity.
Booking Service	8083	Booking lifecycle management (PENDING → CONFIRMED → COMPLETED / CANCELLED), tenant income verification, owner approval with contact exchange.	H2 DB (bookingdb) Entity: Booking	Calls Property Service (port 8082) via REST to validate property & owner.
Payment Service	8084	Token payment (₹5,000 reservation lock), monthly rent payment, mock payment gateway simulation, tenant refund processing.	H2 DB (paymentdb) Entity: Payment	Called by frontend after booking confirmation; verifies tenant ownership.

Crucial Refactoring Note: Plain Java Transition (Removing Lombok)

In our project, we refactored entities and DTOs across all microservices from Project Lombok annotations (`@Data`, `@Builder`, `@NoArgsConstructor`) to **clean, standard Plain Java**. This eliminates Lombok annotation-processor incompatibilities with modern JDK preview features (such as JDK 26 preview / Java 21 toolchains), provides 100% transparent bytecode without hidden synthetic methods, and guarantees rapid, deterministic Maven compilation.

CHAPTER 2: SPRING BOOT FUNDAMENTALS & CORE ANNOTATIONS

1. Inversion of Control (IoC) & Dependency Injection (DI)

In standard Java, an object creates its own dependencies using `new Service()`. This creates tight coupling. With Spring's **Inversion of Control (IoC)** container, control is inverted: the Spring Framework instantiates, configures, and manages the lifecycle of beans. **Dependency Injection (DI)** is the design pattern through which IoC is achieved. Spring injects required dependencies into classes automatically at runtime.

2. Constructor Injection vs Field Injection (Why Field Injection is Bad)

Why we use Constructor Injection across all our Services:

- Field Injection** (`@Autowired private BookingRepository repo;`): Cannot make fields `final`; impossible to instantiate or test without a heavy Spring context; hides dependencies.
- Constructor Injection (Our Approach)**: Fields are declared `private final` (immutable thread safety). In unit tests, we can instantiate the service directly with mock objects using `new BookingService(mockRepo, mockWebClient)` without booting up Spring!

```
// Production Pattern Used in BookingService.java
@Service
public class BookingService {
    private final BookingRepository bookingRepository;
    private final WebClient.Builder webClientBuilder;

    // Constructor Injection - completely testable with pure Mockito
    public BookingService(BookingRepository bookingRepository, WebClient.Builder webClientBuilder) {
        this.bookingRepository = bookingRepository;
        this.webClientBuilder = webClientBuilder;
    }
}
```

3. Spring Boot Annotation Reference Guide

Annotation	Purpose & Mechanism	Project Example
@SpringBootApplication	Meta-annotation combining @Configuration, @EnableAutoConfiguration, and @ComponentScan. Entry point.	BookingServiceApplication.java main class.
@RestController	Combines @Controller and @ResponseBody. Automatically serializes Java return objects into JSON over HTTP.	BookingController, PaymentController.
@Service / @Repository	Stereotype annotations marking business logic beans and database DAO components for Spring component scanning.	MockPaymentService, PaymentRepository.
@GetMapping / @PostMapping	Routes HTTP GET and POST requests to specific controller methods.	@PostMapping("/api/bookings")
@PathVariable	Extracts dynamic values from the URL path.	@GetMapping("/{id}") public BookingResponse get(@PathVariable Long id)
@RequestBody	Deserializes incoming JSON request body into a strongly typed DTO object.	public PaymentResponse pay(@Valid @RequestBody PaymentRequest req)
@Entity & @Table	JPA annotations mapping Java classes to relational database tables.	@Entity @Table(name = "bookings") public class Booking
@PrePersist & @PreUpdate	JPA entity lifecycle callbacks that populate createdAt and updatedAt timestamps before SQL execution.	Auto-sets createdAt = LocalDateTime.now().
@RestControllerAdvice	Global interceptor for handling exceptions across all controllers in a unified manner.	GlobalExceptionHandler.java

CHAPTER 3: DTO PATTERN & INPUT VALIDATION ARCHITECTURE

1. Why DTOs (Data Transfer Objects) are Essential

An **Entity** represents the internal database schema. A **DTO** represents the public contract between client and API. Exposing Entities directly is an anti-pattern because:

- **Mass Assignment Attack:** Malicious users can send JSON payloads containing fields like `role: "ADMIN"` or `status: "CONFIRMED"`. DTOs restrict input to only permitted editable fields.
- **Data Leakage:** Entities often contain internal metadata, audit logs, or password hashes. DTOs ensure only safe attributes are returned.
- **Infinite Recursion:** Jackson JSON serializer can enter infinite circular loops when serializing bidirectional JPA relationships (e.g. User ↔ Booking). DTOs eliminate circular dependencies.
- **Schema Independence:** Database columns can be renamed or refactored without breaking mobile apps or frontend web interfaces.

2. Full DTO Code Implementations in LuxeFlats

A. RegisterRequest.java (Auth Service Input DTO with Regex Validation)

Location: `auth-service/src/main/java/com/flatrental/auth/dto/RegisterRequest.java`

```
package com.flatrental.auth.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

public class RegisterRequest {
    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 50, message = "Username must be between 3 and 50 characters")
    private String username;

    @NotBlank(message = "Email is required")
    @Email(regexp = "^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$", message = "Please provide a valid email address")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 6, message = "Password must be at least 6 characters")
    private String password;

    @NotBlank(message = "Full name is required")
    private String fullName;

    @NotBlank(message = "Phone number is required")
    @Pattern(regexp = "[6-9]\\d{9}$", message = "Phone number must be a valid 10-digit Indian number")
    private String phoneNumber;

    private String role; // Defaults to TENANT if null

    public RegisterRequest() {}
    // Full Getters & Setters implemented in Plain Java...
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }
    public String getPhoneNumber() { return phoneNumber; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

B. BookingRequest.java & BookingResponse.java

Location: `booking-service/src/main/java/com/flatrental/booking/dto/BookingRequest.java`

```

public class BookingRequest {
    @NotNull(message = "Property ID is required")
    private Long propertyId;

    @NotNull(message = "Start date is required")
    private LocalDate startDate;

    @NotNull(message = "End date is required")
    private LocalDate endDate;

    private String tokenPaymentReference;
    // Getters and setters...
}

```

C. PaymentRequest.java

Location: payment-service/src/main/java/com/flatrental/payment/dto/PaymentRequest.java

```

public class PaymentRequest {
    @NotNull(message = "Booking ID is required")
    private Long bookingId;

    @NotNull(message = "Amount is required")
    @DecimalMin(value = "0.01", message = "Amount must be greater than 0")
    private BigDecimal amount;

    @NotNull(message = "Payment method is required")
    private PaymentMethod paymentMethod; // CARD, UPI, NET_BANKING

    private String paymentType; // TOKEN, RENT
    // Getters and setters...
}

```

CHAPTER 4: CENTRALIZED EXCEPTION HANDLING & GLOBAL ERROR CONTRACT

1. The Problem with Distributed try-catch Blocks

Without centralized exception handling, controllers end up polluted with repetitive try-catch blocks. If an unhandled exception bubbles up, Spring Boot emits a default **WhiteLabel Error Page** with raw stack traces, leaking internal class paths and SQL queries to the outside world.

2. The Standard Error Response: ApiError.java

Location: common / <service>/exception/ApiError.java

```
package com.flatrental.booking.exception;

import java.time.LocalDateTime;
import java.util.List;

public class ApiError {
    private int status;
    private String message;
    private LocalDateTime timestamp;
    private List<String> details;

    public ApiError() {
        this.timestamp = LocalDateTime.now();
    }

    public ApiError(int status, String message, List<String> details) {
        this.status = status;
        this.message = message;
        this.timestamp = LocalDateTime.now();
        this.details = details;
    }

    // Full Getters, Setters, and Static Builder...
}
```

3. Complete GlobalExceptionHandler.java Implementation

Location: booking-service/src/main/java/com/flatrental/booking/exception/GlobalExceptionHandler.java

```

package com.flatrental.booking.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.util.List;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // 1. Intercepts Jakarta Bean Validation failures (@Valid on DTO)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidationErrors(MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult().getFieldErrors()
            .stream()
            .map(error → error.getField() + ": " + error.getDefaultMessage())
            .collect(Collectors.toList());

        ApiError apiError = new ApiError(HttpStatus.BAD_REQUEST.value(), "Validation Failed", errors);
        return new ResponseEntity<ApiError>(apiError, HttpStatus.BAD_REQUEST);
    }

    // 2. Intercepts Custom Business Domain Exceptions
    @ExceptionHandler(InvalidBookingException.class)
    public ResponseEntity<ApiError> handleInvalidBooking(InvalidBookingException ex) {
        ApiError apiError = new ApiError(HttpStatus.BAD_REQUEST.value(), ex.getMessage(), List.of(ex.getMessage()));
        return new ResponseEntity<ApiError>(apiError, HttpStatus.BAD_REQUEST);
    }

    // 3. Fallback for unhandled unexpected server errors (500)
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleAllExceptions(Exception ex) {
        ApiError apiError = new ApiError(HttpStatus.INTERNAL_SERVER_ERROR.value(), "An unexpected error occurred",
        List.of(ex.getMessage()));
        return new ResponseEntity<ApiError>(apiError, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

4. Before vs After: HTTP Response Payload Comparison

Without Handler (Default WhiteLabel Leak)	With GlobalExceptionHandler (LuxeFlats Standard)
<pre> { "timestamp": "2026-09-14T08:00:00.123", "status": 500, "error": "Internal Server Error", "trace": "java.lang.NullPointerException: Cannot invoke LocalDate.isAfter() ... \n at com.flatrental.booking.service.BookingService.createBookin g() ... ", "path": "/api/bookings" } </pre>	<pre> { "status": 400, "message": "Start date and end date are required", "timestamp": "2026-09-14T08:00:00.123", "details": ["Start date and end date are required"] } </pre>

CHAPTER 5: PROFESSIONAL GIT ARCHITECTURE & RESET VS REVERT

1. Git's 4 Logical Areas & Lifecycle Flow

Git does not simply track files; it operates across 4 distinct memory and disk boundaries:

- **Working Directory:** The local sandbox on your filesystem where you actively write and edit code files.
- **Staging Area (Index):** The intermediate cache where changes are reviewed and prepared using `git add <file>` before being permanently committed.
- **Local Repository (.git):** The local cryptographic commit graph stored on your computer after executing `git commit -m "feat: add null validation"`.
- **Remote Repository (GitHub / GitLab):** The shared central server where team members push and sync branches using `git push origin feature-branch`.

2. Enterprise Branching Strategy

Branch	Lifecycle & Protection	Engineering Purpose & Rules
main / master	Protected. Direct push strictly forbidden.	Contains production-ready code deployed to live AWS/production servers. Merges only allowed via approved Pull Requests.
develop	Integration branch.	Aggregates completed features from multiple developers for staging integration testing.
feature/<name>	Short-lived developer branches.	Example: feature/booking-null-validation. Created from develop, tested thoroughly, and merged via PR.
hotfix/<name>	Emergency branch.	Branched directly from main to fix critical production bugs, then cherry-picked/merged into both main and develop.

3. Resolving Merge Conflicts Step-by-Step

Merge conflicts arise when Git cannot automatically reconcile concurrent line modifications across branches. Git inserts conflict markers directly into the file:

```
<<<<<<< HEAD (Current Branch - e.g., main)
if (request.getStartDate() == null || request.getEndDate() == null) {
    throw new InvalidBookingException("Dates are mandatory");
}
=====
if (request.getStartDate() == null || request.getEndDate() == null) {
    throw new InvalidBookingException("Start date and end date are required");
}
>>>>>>> feature/booking-validation (Incoming Branch)
```

Resolution Steps: (1) Open file and discuss with author, (2) Keep desired code and delete markers (<<<<<<<, =====, >>>>>>>), (3) Run `git add .`, (4) Finalize with `git commit -m "fix: resolve booking validation merge conflict"`.

4. Git Reset vs Git Revert: The Definitive Comparison

The Accounting Ledger Analogy (Master this for the Viva!)

Imagine a corporate bank ledger where an erroneous entry was recorded: "Paid ₹10,000 to John".

- **git reset (Tearing the Page Out):** Rewinds the timeline and erases history as if the transaction never occurred. In accounting, tearing out a ledger page is an audit violation. If teammates have already copied the ledger, their copies will conflict!
- **git revert (Adding an Adjustment Entry):** Adds a new entry at the bottom of the ledger: "Reversal adjustment: Refunded ₹10,000 from John". History is preserved, audits pass cleanly, and team collaboration is 100% safe.

Command	Working Directory Impact	Staging Area Impact	When to Use (Industry Rule)
<code>git reset --soft HEAD~1</code>	Changes kept untouched.	Changes remain staged!	Local only. Modifying commit message or adding forgotten files into previous commit.
<code>git reset --mixed HEAD~1</code> (Default)	Changes kept untouched in files.	Changes un-staged.	Local only. Splitting a large commit into smaller commits.
<code>git reset --hard HEAD~1</code>	Destroys all file changes!	Destroyed!	Local only. Discarding unwanted experimental code permanently. Never use on pushed commits!
<code>git revert <commit-hash></code>	Applies inverse changes safely.	Stages inverse diff.	MANDATORY for shared/pushed commits on GitHub. Non-destructive; creates a new commit undoing the mistake.

CHAPTER 6: UNIT TESTING MASTERCLASS WITH JUNIT 5 & MOCKITO

1. What is a Unit Test and Why do we Mock?

A **Unit Test** validates the behavior of the smallest testable piece of code (typically a single method) in total isolation from external systems such as databases, network sockets, or external microservices.

- **Why not Integration Tests for everything?** Running the full Spring Boot application (`@SpringBootTest`) takes 15–30 seconds per test because it boots Hibernate, embeds an H2 database, and scans beans. In contrast, pure Mockito unit tests execute in **5 to 10 milliseconds!**
- **The AAA (Arrange-Act-Assert) Pattern:**
 1. **Arrange:** Prepare mock dependencies, mock data, and define stub behaviors using `when().thenReturn()`.
 2. **Act:** Invoke the exact business method under test (e.g. `bookingService.createBooking(...)`).
 3. **Assert:** Verify expected outcomes using `assertEquals()`, `assertThrows()`, and behavior verification with `verify()`.

2. Core Mockito Annotations & Methods

Annotation / Method	Underlying Mechanism	Project Usage
<code>@ExtendWith(MockitoExtension.class)</code>	JUnit 5 extension that initializes Mockito annotations and validates framework rules before each test.	Annotated on top of <code>BookingServiceTest</code> and <code>MockPaymentServiceTest</code> .
<code>@Mock</code>	Creates a simulated dummy instance of a dependency (returns defaults like null or 0 unless stubbed).	<code>@Mock private BookingRepository bookingRepository;</code>
<code>@InjectMocks</code>	Instantiates the real service under test and injects all created <code>@Mock</code> objects into its constructor.	<code>@InjectMocks private BookingService bookingService;</code>
<code>when(...).thenReturn(...)</code>	Configures stub behavior on mock methods.	<code>when(bookingRepository.findById(1L)).thenReturn(Optional.of(sampleBooking));</code>
<code>assertThrows(...)</code>	Asserts that a specific exception class is thrown when the executable lambda runs.	<code>assertThrows(InvalidBookingException.class, () -> service.createBooking(req, 1L));</code>
<code>verify(mock, never()).method(...)</code>	Behavior verification: guarantees that a method was NEVER invoked during the execution path.	<code>verify(bookingRepository, never()).save(any(Booking.class));</code>

CHAPTER 7: BOOKING SERVICE UNIT TESTING DEEP-DIVE

Booking Service Test File Coordinates

Test File: booking-service/src/test/java/com/flatrental/booking/service/BookingServiceTest.java

Target Class: booking-service/src/main/java/com/flatrental/booking/service/BookingService.java

Total Tests: 13 Tests (11 Baseline Tests + 2 Newly Engineered Defensive Tests) • **Pass Rate: 100%**

1. Breakdown of All 13 Tests in BookingServiceTest

#	Test Method Name	Type	Scenario & Verification Logic
1	testCreateBooking_Success	Baseline	Valid dates; verifies booking is saved with status PENDING.
2	testCreateBooking_WithTokenPaymentReference	Baseline	Valid request with token transaction reference attached; verifies reference is persisted.
3	testCreateBooking_InvalidDates	Baseline	End date is before start date; verifies InvalidBookingException is thrown.
4	testGetBookingById_Success	Baseline	Queries existing booking; verifies mapped BookingResponse matches repository entity.
5	testGetBookingById_NotFound	Baseline	Queries non-existent ID; verifies InvalidBookingException("Booking not found").
6	testUpdateStatus_ValidTransition	Baseline	Transitions status from PENDING → CONFIRMED; verifies update saved.
7	testUpdateStatus_AlreadyCancelled_ThrowsException	Baseline	Attempts transition on already-cancelled booking; verifies rejection exception.
8	testCancelBooking_Success	Baseline	Tenant cancels their own pending booking; verifies status set to CANCELLED.
9	testSubmitVerification_InvalidIncome	Baseline	Tenant submits monthly income less than 3x rent; verifies verification failure.
10	testApproveWithContactShare_Success	Baseline	Owner approves booking and shares contact details; verifies property owner validation passes.
11	testApproveWithContactShare_Unauthorized	Baseline	User other than property owner attempts approval; verifies UnauthorizedException.
12	testCreateBooking_NullStartDate_ThrowsException	ADDED TEST	Passes startDate = null. Asserts InvalidBookingException; verifies save() never called.
13	testCreateBooking_NullEndDate_ThrowsException	ADDED TEST	Passes endDate = null. Asserts InvalidBookingException; verifies save() never called.

2. Complete Source Code of the 2 Added Tests

```
@Test
@DisplayName("Should reject booking when start date is missing (null)")
void testCreateBooking_NullStartDate_ThrowsException() {
    // Arrange
    BookingRequest request = new BookingRequest();
    request.setPropertyId(100L);
    request.setStartDate(null); // Missing start date
    request.setEndDate(LocalDate.of(2026, 9, 30));

    // Act & Assert: Must throw InvalidBookingException, NOT NullPointerException
    InvalidBookingException exception = assertThrows(
        InvalidBookingException.class,
        () -> bookingService.createBooking(request, 200L)
    );

    assertEquals("Start date and end date are required", exception.getMessage());
    // Guarantee repository save was NEVER reached
    verify(bookingRepository, never()).save(any(Booking.class));
}

@Test
@DisplayName("Should reject booking when end date is missing (null)")
void testCreateBooking_NullEndDate_ThrowsException() {
    // Arrange
    BookingRequest request = new BookingRequest();
    request.setPropertyId(100L);
    request.setStartDate(LocalDate.of(2026, 9, 1));
    request.setEndDate(null); // Missing end date

    // Act & Assert
    InvalidBookingException exception = assertThrows(
        InvalidBookingException.class,
        () -> bookingService.createBooking(request, 200L)
    );

    assertEquals("Start date and end date are required", exception.getMessage());
    verify(bookingRepository, never()).save(any(Booking.class));
}
```

3. Initial Failure, Root Cause & Production Bug Fix

The Initial Test Failure

Expected: com.flatrental.booking.exception.InvalidBookingException

Actual: java.lang.NullPointerException: Cannot invoke "java.time.LocalDate.isAfter(java.time.chrono.ChronoLocalDate)" because the return value of "request.getEndDate()" is null

Root Cause: Line 49 of BookingService.java directly executed date comparison:

```
if (!request.getEndDate().isAfter(request.getStartDate()))
```

When endDate was null, invoking .isAfter(...) threw an unhandled NPE. When startDate was null, passing null into isAfter(null) also threw an NPE!

Before vs After Code Diff in BookingService.java (Lines 46-55):

```
- // BEFORE (VULNERABLE TO NULL POINTER EXCEPTIONS):  
  
- if (!request.getEndDate().isAfter(request.getStartDate())) {  
-     throw new InvalidBookingException("End date must be after start date");  
- }  
  
+ // AFTER (DEFENSIVE VALIDATION FIX):  
+ if (request == null) {  
+     throw new InvalidBookingException("Booking request cannot be null");  
+ }  
+ if (request.getStartDate() == null || request.getEndDate() == null) {  
+     throw new InvalidBookingException("Start date and end date are required");  
+ }  
+ if (!request.getEndDate().isAfter(request.getStartDate())) {  
+     throw new InvalidBookingException("End date must be after start date");  
+ }
```

CHAPTER 8: PAYMENT SERVICE UNIT TESTING DEEP-DIVE

Payment Service Test File Coordinates

Test File: payment-service/src/test/java/com/flatrental/payment/service/MockPaymentServiceTest.java

Target Class: payment-service/src/main/java/com/flatrental/payment/service/MockPaymentService.java

Total Tests: 13 Tests (10 Baseline Tests + 3 Newly Engineered Security Tests) • **Pass Rate: 100%**

1. Breakdown of All 13 Tests in MockPaymentServiceTest

#	Test Method Name	Type	Scenario & Verification Logic
1	testCreatePayment_Token_Success	Baseline	Initial ₹5,000 token payment; verifies saved as COMPLETED with generated txn ref.
2	testCreatePayment_Token_AlreadyExists_ThrowsException	Baseline	Attempts duplicate token payment for same booking; verifies duplicate rejection.
3	testCreatePayment_Rent_Success	Baseline	Rent payment after token payment; verifies successful completion.
4	testGetPaymentById_Success	Baseline	Queries existing payment ID; asserts mapped response match.
5	testGetPaymentById_NotFound	Baseline	Queries non-existent ID; asserts PaymentNotFoundException.
6	testGetPaymentsByBookingId_Success	Baseline	Queries all payments belonging to a booking; returns list of payments.
7	testGetPaymentsByTenantId_Success	Baseline	Queries payment history for a specific tenant ID.
8	testUpdatePaymentStatus_Success	Baseline	Updates payment status; verifies modified timestamp and status change.
9	testInitiateRefund_Success	Baseline	Owner/Tenant initiates refund on completed payment; refund service called.
10	testInitiateRefund_NotCompleted_ThrowsException	Baseline	Attempts refund on pending or failed payment; verifies rejection.
11	testCreatePayment_ZeroAmount_ThrowsException	ADDED TEST	Passes amount = BigDecimal.ZERO; asserts PaymentProcessingException.
12	testCreatePayment_NegativeAmount_ThrowsException	ADDED TEST	Passes amount = new BigDecimal("-5000.00"); asserts PaymentProcessingException.
13	testInitiateRefund_WrongUser_ThrowsException	ADDED TEST	Tenant #999 attempts refunding payment owned by Tenant #200; asserts unauthorized rejection.

2. Complete Source Code of the 3 Added Tests

```
@Test
@DisplayName("Should reject payment when amount is zero")
void testCreatePayment_ZeroAmount_ThrowsException() {
    // Arrange: Zero amount
    PaymentRequest request = new PaymentRequest();
    request.setBookingId(100L);
    request.setAmount(BigDecimal.ZERO);
    request.setPaymentMethod(PaymentMethod.CARD);
    request.setPaymentType("TOKEN");

    // Act & Assert
    PaymentProcessingException ex = assertThrows(
        PaymentProcessingException.class,
        () -> paymentService.createPayment(request, 200L)
    );

    assertEquals("Payment amount must be greater than zero.", ex.getMessage());
    verify(paymentRepository, never()).save(any(Payment.class));
}

@Test
@DisplayName("Should reject payment when amount is negative")
void testCreatePayment_NegativeAmount_ThrowsException() {
    // Arrange: Negative amount (-₹5,000)
    PaymentRequest request = new PaymentRequest();
    request.setBookingId(100L);
    request.setAmount(new BigDecimal("-5000.00"));
    request.setPaymentMethod(PaymentMethod.CARD);
    request.setPaymentType("TOKEN");

    // Act & Assert
    PaymentProcessingException ex = assertThrows(
        PaymentProcessingException.class,
        () -> paymentService.createPayment(request, 200L)
    );

    assertEquals("Payment amount must be greater than zero.", ex.getMessage());
    verify(paymentRepository, never()).save(any(Payment.class));
}

@Test
@DisplayName("Should reject refund when current user is not the payment owner")
void testInitiateRefund_WrongUser_ThrowsException() {
    // Arrange: Payment belongs to Tenant 200L, caller is Tenant 999L
    when(paymentRepository.findById(1L)).thenReturn(Optional.of(sampleTokenPayment));

    // Act & Assert
    PaymentProcessingException ex = assertThrows(
        PaymentProcessingException.class,
        () -> paymentService.initiateRefund(1L, 999L)
    );

    assertEquals("You are not authorized to refund this payment.", ex.getMessage());
    verify(refundService, never()).processRefund(any(Payment.class));
}
```

3. Critical Production Bugs & Code Fixes

Critical Security & Financial Flaws Uncovered by Unit Tests:

- **Flaw 1 (Zero & Negative Amounts):** `createPayment()` accepted zero or negative values. A user could pay ₹0 or -₹5,000 and the system saved the status as `COMPLETED`, locking properties for free or corrupting account books!
- **Flaw 2 (IDOR Vulnerability in Refunds):** `initiateRefund(paymentId, currentUserId)` took `currentUserId` as a parameter but never verified if it matched `payment.getTenantId()`! Any logged-in user could trigger refunds on other people's payments!

Production Fixes in MockPaymentService.java:

```
// 1. Fix for Zero/Negative Amounts (Lines 30-34):
if (request.getAmount() == null || request.getAmount().compareTo(BigDecimal.ZERO) ≤ 0) {
    throw new PaymentProcessingException("Payment amount must be greater than zero.");
}

// 2. Fix for Unauthorized Refund IDOR (Lines 108-111):
if (!payment.getTenantId().equals(currentUserId)) {
    throw new PaymentProcessingException("You are not authorized to refund this payment.");
}
```

Advanced Mockito Insight: UnnecessaryStubbingException

During test authoring, our initial zero-amount test contained:

```
when(paymentRepository.findByBookingId(100L)).thenReturn(Collections.emptyList());
```

Mockito threw `org.mockito.exceptions.misusing.UnnecessaryStubbingException`. **Why?** Because our newly added validation guard throws *before* the code ever reaches `findByBookingId`! Mockito strict stubbing detects that the stub was never invoked. Removing the stub proved that the early exit guard works perfectly.

CHAPTER 9: AUTH & PROPERTY SERVICES VALIDATION & CONTROLLER TESTS

1. Auth Service Testing Strategy: Direct Validator vs MockMvc

When testing Spring Boot controllers, testing `AuthService.registerUser(request)` directly in unit tests **bypasses** Jakarta Bean Validation annotations (such as `@NotBlank`, `@Email`, and `@Pattern`). To achieve 100% test coverage, we created two complementary test suites:

- **RegisterRequestValidationTest.java (23 Tests):** Direct unit testing of the Jakarta Validator instance to test every edge case of phone regex (`^[6-9]\d{9}$`) and email format in isolation.
- **AuthControllerValidationTest.java (5 Tests):** Full HTTP mock testing with Spring MockMvc verifying that invalid payloads result in HTTP 400 Bad Request and that the database repository is never invoked.

2. Auth Controller MockMvc Test Code

Location: `auth-service/src/test/java/com/flatrental/auth/controller/AuthControllerValidationTest.java`

```
@WebMvcTest(AuthController.class)
public class AuthControllerValidationTest {
    @Autowired private MockMvc mockMvc;
    @MockBean private AuthService authService;
    @Autowired private ObjectMapper objectMapper;

    @Test
    @DisplayName("HTTP 400 when registration request has invalid phone number")
    void testRegister_InvalidPhone_Returns400() throws Exception {
        RegisterRequest req = new RegisterRequest();
        req.setUsername("rahul123");
        req.setEmail("rahul@example.com");
        req.setPassword("pass1234");
        req.setFullName("Rahul Sharma");
        req.setPhoneNumber("1234567890"); // Invalid: does NOT start with 6-9!

        mockMvc.perform(post("/api/auth/register")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.message").value("Validation Failed"))
            .andExpect(jsonPath("$.details[0]").value(containsString("valid 10-digit Indian number")));

        // Verify service was never reached
        verify(authService, never()).registerUser(any());
    }
}
```

3. Property Service Test Suite (16 Tests)

Location: `property-service/src/test/java/com/flatrental/property/service/PropertyServiceTest.java`

- **Image Parsing Logic:** Validates pipe-delimited image URL extraction ("`https://img1.jpg|https://img2.jpg`") ensuring fallback behavior when images are empty.
- **Availability Filtering:** Tests active property search filtering (`findByAvailableTrue()` vs city/price queries).
- **Ownership Verification:** Rejects modification or deletion attempts when the requesting user's ID does not match the property's `ownerId`.

CHAPTER 10: MASTER PROJECT REVIEW & VIVA Q&A CHEAT SHEET

Use these script-ready answers word-for-word in front of your examiner or interviewer. Hinglish intuition is provided to cement your understanding!

Q1: "Can you give a technical high-level overview of your project architecture?"

Your Answer: "Sir, LuxeFlats is a real-estate rental marketplace engineered with a Spring Boot 3 microservices architecture following the Database-per-Service pattern. We have an API Gateway on port 8080 acting as a reverse proxy, routing to Auth Service on 8081 for JWT authentication, Property Service on 8082 for rental listings, Booking Service on 8083 for lease lifecycle management, and Payment Service on 8084 for token reservations and rent payments. Services communicate via RESTful contracts. Furthermore, we designed all entities and DTOs in standard Plain Java without Lombok to ensure complete JDK 26 preview compatibility and transparent bytecode."

Hinglish Samajh: Seedha bolo ki project Spring Boot microservices par hai, har service ka alag database hai (Database-per-Service). Gateway port 8080 par hai jo baki 4 services (Auth, Property, Booking, Payment) ko route karta hai. Sab REST se judte hain aur Lombok hata ke clean Java use kiya hai.

Q2: "Why do you use DTOs instead of passing JPA Entities directly to the frontend?"

Your Answer: "Sir, passing JPA Entities directly exposes the internal database schema and creates severe security vulnerabilities like Mass Assignment—where an attacker could inject an administrative role or modify payment status. DTOs decouple our internal persistence model from our external API contract. They also prevent Jackson recursive serialization issues in bidirectional relationships and allow us to isolate Jakarta Bean Validation rules on incoming payloads."

Hinglish Samajh: Entity DB table hai, DTO bahar walo ke liye contract hai. Agar direct entity doge toh koi role='ADMIN' bhejke hack kar sakta hai (mass assignment) ya password hash leak ho jayega. DTO sirf wahi fields leti/deti hai jo zaroori hain.

Q3: "How does Centralized Exception Handling work in your microservices?"

Your Answer: "We implement centralized exception handling using `@RestControllerAdvice` with custom `@ExceptionHandler` methods in `GlobalExceptionHandler.java`. Instead of writing try-catch blocks in each controller, unhandled domain exceptions like `InvalidBookingException` or validation failures like `MethodArgumentNotValidException` are intercepted globally. We translate them into a standardized `ApiError` JSON payload with HTTP status codes like 400 Bad Request, hiding stack traces from end-users."

Hinglish Samajh: Har controller me try-catch lagane ke bajay ek global class banayi `@RestControllerAdvice` wali. Koi bhi error aaye, ye pakadti hai aur clean JSON (`ApiError`) 400 status ke sath bhejti hai, jisse default `WhiteLabel` error page aur internal details leak na ho.

Q4: "Did you write unit tests? What testing stack did you use?"

Your Answer: "Yes, sir. We authored a comprehensive 89-test unit testing suite across our services with a 100% pass rate. We utilized **JUnit 5** as our testing engine and **Mockito** for mock creation and stubbing. In our controllers, we used Spring's `MockMvc` to test HTTP endpoints without booting a heavy servlet container. We follow the Arrange-Act-Assert (AAA) pattern for test structure."

Hinglish Samajh: Bolna 89 unit tests likhe hain 100% pass rate ke sath. JUnit 5 aur Mockito use kiya hai. Har test 5 se 10 millisecond me chalta hai bina poora Spring Boot boot kiye.

Q5: "What real bugs did your unit tests uncover in Booking Service?"

Your Answer: "In `BookingServiceTest`, when we added boundary tests for null start and end dates (`testCreateBooking_NullStartDate_ThrowsException`), both tests failed with an unhandled `NullPointerException` instead of our domain `InvalidBookingException`. The root cause was that line 49 in `BookingService.java` called `request.getEndDate().isAfter(request.getStartDate())` without prior null checks. We fixed this by introducing defensive guard clauses validating null request and null dates before the date comparison."

Hinglish Samajh: Booking me jab start ya end date null aati thi, toh code direct `.isAfter()` call karta tha jisse `NullPointerException` aati thi. Hamne unit test likha, fail hua, fir code me null checks lagaye date comparison se pehle.

Q6: "What critical bugs did you catch in Payment Service?"

Your Answer: "Our unit tests uncovered two severe flaws in `MockPaymentService.java`. First, the service allowed ₹0 and negative payment amounts to be processed and marked as `COMPLETED`. Second, the `initiateRefund()` method failed to check if the caller was the actual owner of the payment, creating an IDOR authorization vulnerability. We added early-exit domain guards checking `amount.compareTo(BigDecimal.ZERO) <= 0` and verifying `payment.getTenantId().equals(currentUserId)`."

Hinglish Samajh: Payment service me do bade bug pakde: Pehla, log 0 ya negative rupee dekar bhi payment complete kar lete the. Doosra, koi bhi user kisi aur ka payment refund karwa sakta tha. Hamne unit test banaye aur service me strict amount aur tenant authorization checks lagaye.

Q7: "Explain Mockito Strict Stubbing and UnnecessaryStubbingException."

Your Answer: "Mockito 3+ enforces Strict Stubbing to prevent dead code in test suites. If a test stubs a method using `when(mock.method()).thenReturn(...)` but the execution path exits before reaching that method, Mockito throws `UnnecessaryStubbingException`. We experienced this in our zero-amount payment test: because our validation guard exited early, the repository mock was never called. Removing the unnecessary stub verified that our early exit guard was working as intended."

Hinglish Samajh: Agar tumne mock banaya lekin code wahan tak pahucha hi nahi (pehle hi exception fek di), toh Mockito `UnnecessaryStubbingException` fekta hai. Isse prove hota hai ki hamara validation check code ke shuru me hi rok deta hai.

Q8: "What is the exact technical difference between git reset and git revert?"

Your Answer: "Git reset rewinds the branch HEAD pointer and modifies past commit history, which can permanently delete commits (especially with `--hard`). Because it rewrites history, it is only safe for local, unpushed branches. In contrast, `git revert` is non-destructive: it calculates the inverse diff of a commit and appends a brand new commit to the timeline. It preserves complete history and is the industry standard for undoing changes on shared, pushed remote branches."

Hinglish Samajh: Bank ledger yaad rakho! Reset ka matlab panna faadna (history delete karna) - dangerous agar sabke paas copy ho. Revert ka matlab nayi entry daal ke hisaab barabar karna - 100% safe aur professional standard jo sabke sath sync rehta hai.